CERTIFIED ETHICAL HACKER • FINAL PROJECT 5

# Mini SIEM

*Security Information & Event Management — Live Attack Detection & MITRE ATT&CK Mapping*

| Field | Detail |
|---|---|
| Project | Project 5 — Mini SIEM (Security Information & Event Management) |
| Author | Anwar |
| Host / Analyst Machine | kalibaba@Kalibaba (Kali Linux) |
| Attacker VM | Kali Linux — 192.168.229.159 |
| Target / Secondary VM | Windows 10 — 192.168.229.164 |
| Platform | VMware Workstation |
| Language / Dependencies | Python 3 (standard library only) |
| Date | July 2026 |

# Contents

## 1. Executive Summary

This project delivers a working, lightweight Security Information and Event Management (SIEM) system built entirely in Python 3 using only the standard library. The tool ingests real security logs from three independent sources on a live Kali Linux host — the SSH service, the Apache web server, and the UFW firewall — normalises every record into a single common event schema, and applies a set of correlation rules that identify multi-stage attacks rather than isolated log lines.

To validate the SIEM against genuine data rather than synthetic samples, a controlled three-stage attack chain was executed against the host: an SSH brute-force attempt, a web vulnerability scan combined with injection payloads, and a network port scan launched from the Windows 10 virtual machine. Every rule that fired was automatically mapped to the corresponding MITRE ATT&CK technique and tactic, and the results were rendered into a professional HTML dashboard.

The deployed system processed **7,537 real events** across the three log sources and generated **14 correlated alerts** driven by **five correlation rules mapped to four MITRE ATT&CK techniques** (T1110, T1046, T1190 and T1595 — T1110 covering both the attempted and the successful credential attack). The project also surfaced and resolved two authentic engineering problems — a virtual-host logging mismatch and classic SIEM alert-flooding — both documented in Section 8.

| 7,537 | 14 | 13 | 7 |
|---|---|---|---|
| EVENTS PROCESSED | ALERTS RAISED | CRITICAL / HIGH | UNIQUE SOURCE IPS |

## 2. Project Objectives

The objectives of this project were to:

- Collect security-relevant logs from multiple, heterogeneous sources on a live system.
- Normalise disparate log formats into a single, consistent event schema.
- Correlate normalised events using detection rules that recognise attacker behaviour, not just single log entries.
- Map every generated alert to the MITRE ATT&CK framework to provide analyst context.
- Validate the system end-to-end against a real, self-generated attack chain.
- Present findings through a clear, professional dashboard suitable for a security operations context.

## 3. Lab Environment

All work was performed inside an isolated VMware Workstation lab. Table 1 lists the hosts and their roles.

| Host | IP Address | Role |
|---|---|---|
| Kali Linux (Kalibaba) | 192.168.229.159 | SIEM analyst host; also used to launch SSH and web attacks |

| Host | IP Address | Role |
|---|---|---|
| Windows 10 x64 | 192.168.229.164 | Secondary host; launched the network port scan |
| VMware Workstation | — | Virtualisation platform hosting both VMs on one NAT segment |

**Table 1: Lab environment and host roles**

## 4. SIEM Architecture

The SIEM follows the same five-stage pipeline used by production platforms, scaled down to a single, readable Python script. Each stage feeds the next:

| Stage | Name | Function |
|---|---|---|
| 1 | Collection | Read raw log lines from /var/log/auth.log (SSH), the Apache virtual-host access log, and /var/log/ufw.log (firewall). |
| 2 | Normalisation | Parse each source's distinct format into one common event dictionary (timestamp, source, event type, source IP, detail, severity). |
| 3 | Correlation | Group events by source IP and apply detection rules that recognise brute force, port scanning, web scanning, and web attacks. |
| 4 | MITRE Mapping | Attach the relevant ATT&CK tactic and technique ID to every alert. |
| 5 | Reporting | Render an HTML dashboard with summary metrics, an alert table, log-source totals, and top source IPs. |

**Table 2: The five-stage SIEM processing pipeline**

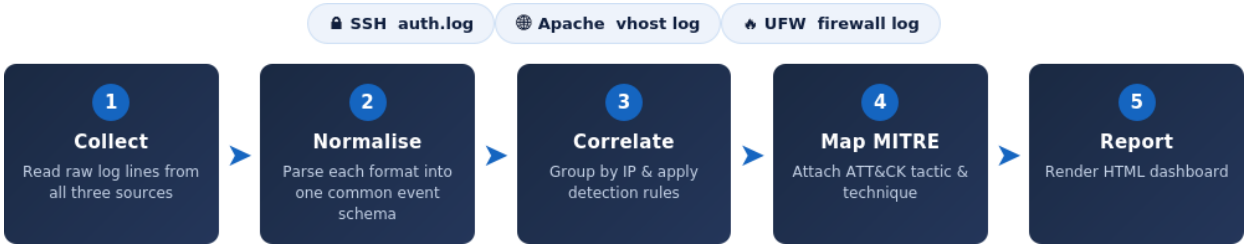


**Figure 1: The five-stage SIEM pipeline, from log collection to dashboard**

Because the tool depends only on the Python standard library, it runs on any stock Kali installation without additional packages — an intentional design choice that keeps the analyst host clean and the code fully auditable.

## 5. Detection Rules

Five correlation rules were implemented. Each is deliberately threshold-based and self-explanatory so its logic can be defended and tuned. Thresholds are defined as constants at the top of the script.

| Rule | Trigger Condition | Severity |
|---|---|---|
| SSH brute force | ≥ 5 failed SSH logins from one source IP | High |

| Rule | Trigger Condition | Severity |
|---|---|---|
| Brute-force success | A successful SSH login from an IP that first produced a burst of failures | Critical |
| Port scan | One IP blocked by the firewall across ≥ 10 different destination ports | High |
| Web scan | A known scanner user-agent (nikto, sqlmap, etc.) or a large volume of 404 responses from one IP | High / Medium |
| Web attack | SQL injection, path traversal, XSS or command-injection patterns in the request URL | Critical |

**Table 3: SIEM correlation rules and their thresholds**

## 6. MITRE ATT&CK Mapping

Mapping each alert to MITRE ATT&CK gives an analyst immediate context: which stage of an intrusion the activity represents and how to prioritise a response. The SIEM maintains a built-in reference table applied automatically during correlation.

| Detection | Technique | Tactic |
|---|---|---|
| SSH brute force | T1110 | Credential Access |
| Brute-force success | T1110 | Credential Access |
| Port scan | T1046 | Discovery |
| Web scanning | T1595 | Reconnaissance |
| Web attack (SQLi / traversal / XSS / cmd) | T1190 | Initial Access |

**Table 4: Mapping of SIEM detections to MITRE ATT&CK**

# 7. Methodology — Attack Simulation

To exercise the SIEM with authentic data, three attacks were launched against the Kali host. The services were first started and the firewall configured to allow SSH (22) and HTTP (80) while blocking and logging all other inbound ports — this arrangement lets brute-force and web attacks reach their services while a port scan is dropped and logged by UFW.

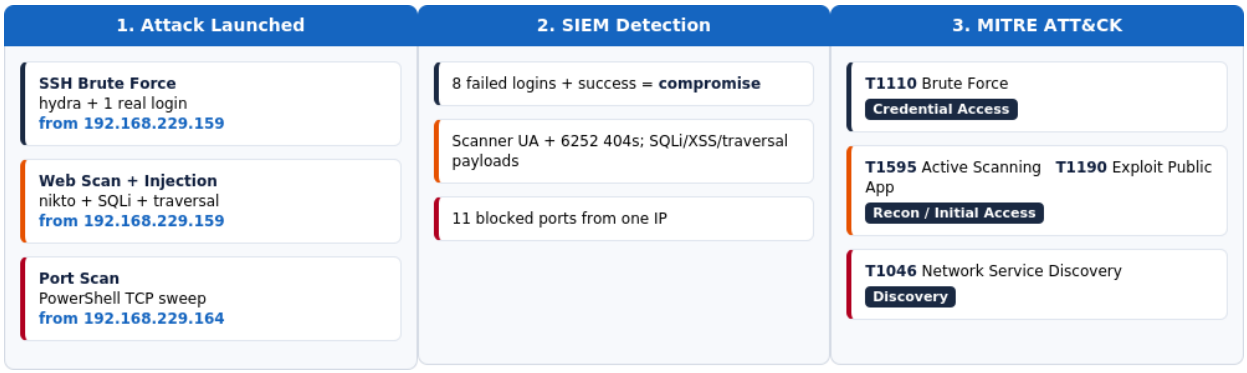


**Figure 2: The three-stage attack chain and how each maps to SIEM detection and MITRE ATT&CK**

## 7.1 Service and Firewall Preparation

```
# Start the logging service and the target services
sudo apt install -y rsyslog
sudo systemctl enable --now rsyslog
sudo systemctl start ssh apache2

# Allow SSH + HTTP; block and log all other inbound ports
sudo ufw allow 22/tcp
sudo ufw allow 80/tcp
sudo ufw default deny incoming
sudo ufw logging on
sudo ufw enable
```

## 7.2 Attack A — SSH Brute Force (T1110)

A short wordlist of common passwords was used with hydra to brute-force the SSH service, followed by a single genuine login. The burst of failures followed by a success reproduces the pattern of a compromised account.

```
hydra -l kalibaba -P ~/siem_project/passlist.txt ssh://192.168.229.159 -t 4 -V
ssh kalibaba@192.168.229.159      # one real login after the failures
```

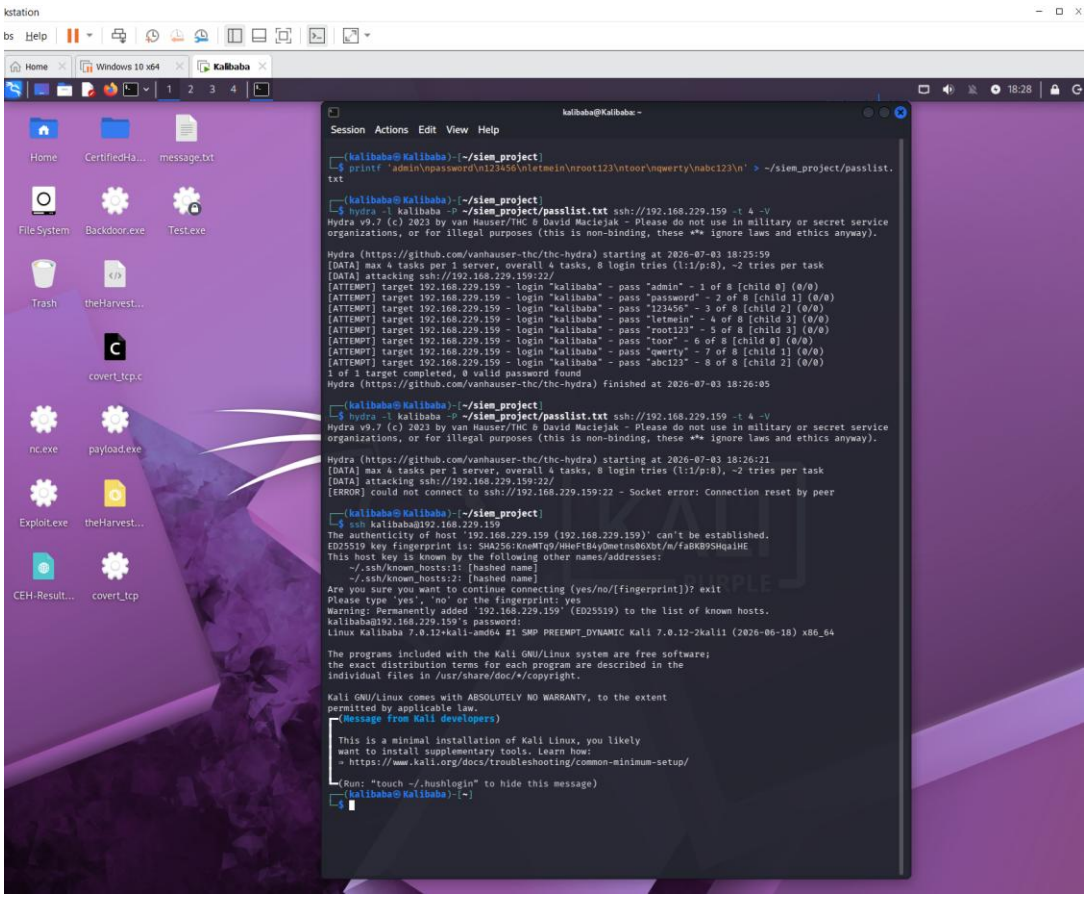


**Figure 3: SSH brute force with hydra (8 failed attempts) followed by a successful login**

Observation worth noting: a second, rapid hydra run was rejected with *"Connection reset by peer"*. This is the SSH daemon's own rate-limiting (MaxStartups) throttling the flood — evidence that the target's built-in defences began dropping the attack, an authentic real-world behaviour.

## 7.3 Attack B — Web Scan and Injection (T1595 / T1190)

The Apache server was scanned with nikto, then three classic web-attack payloads were delivered with curl (URL-encoded, as a real tool would send them):

```
nikto -h http://192.168.229.159 -maxtime 60s
curl -s "http://192.168.229.159/products.php?id=1%27%20OR%20%271%27%3D%271" -o /dev/null
curl -s "http://192.168.229.159/products.php?id=-1%20UNION%20SELECT%20user,pass%20FROM%20users" -o /dev/null
curl -s "http://192.168.229.159/%2e%2e/%2e%2e/%2e%2e/etc/passwd" -o /dev/null
```

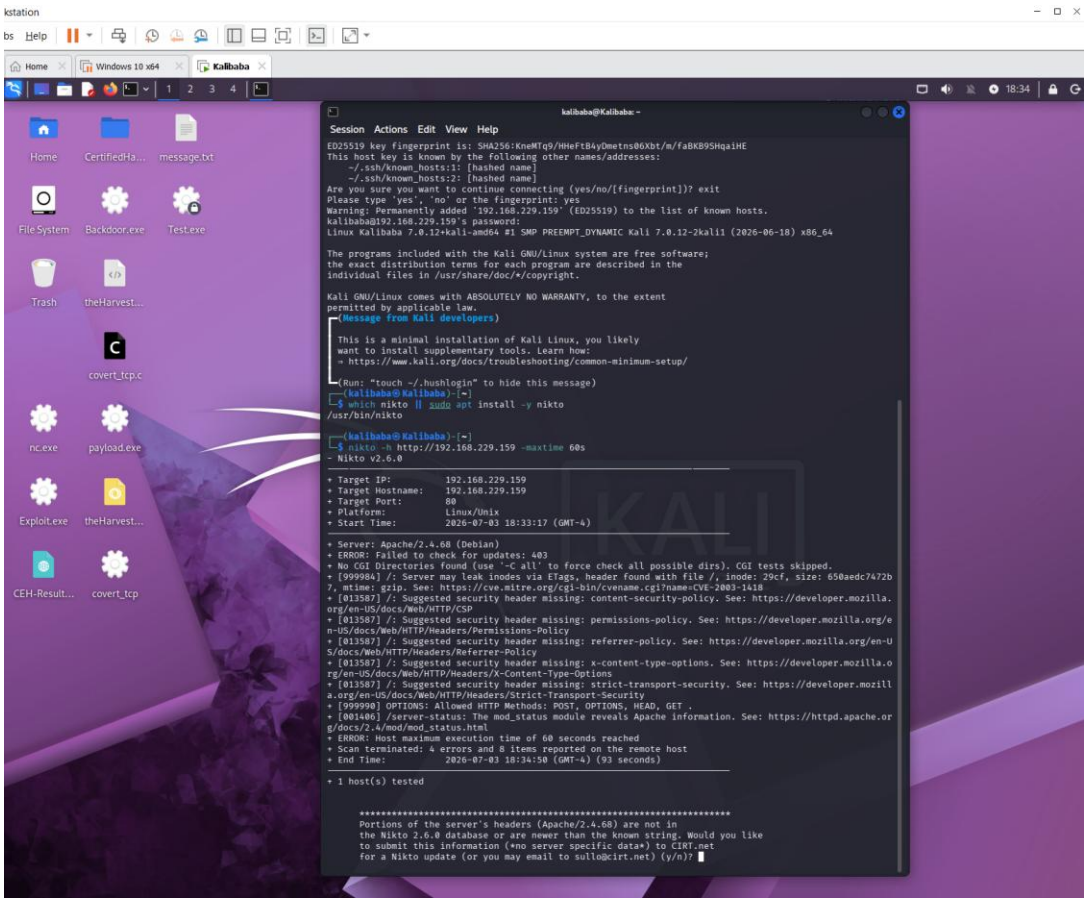


**Figure 4: Nikto web vulnerability scan against the Apache server**

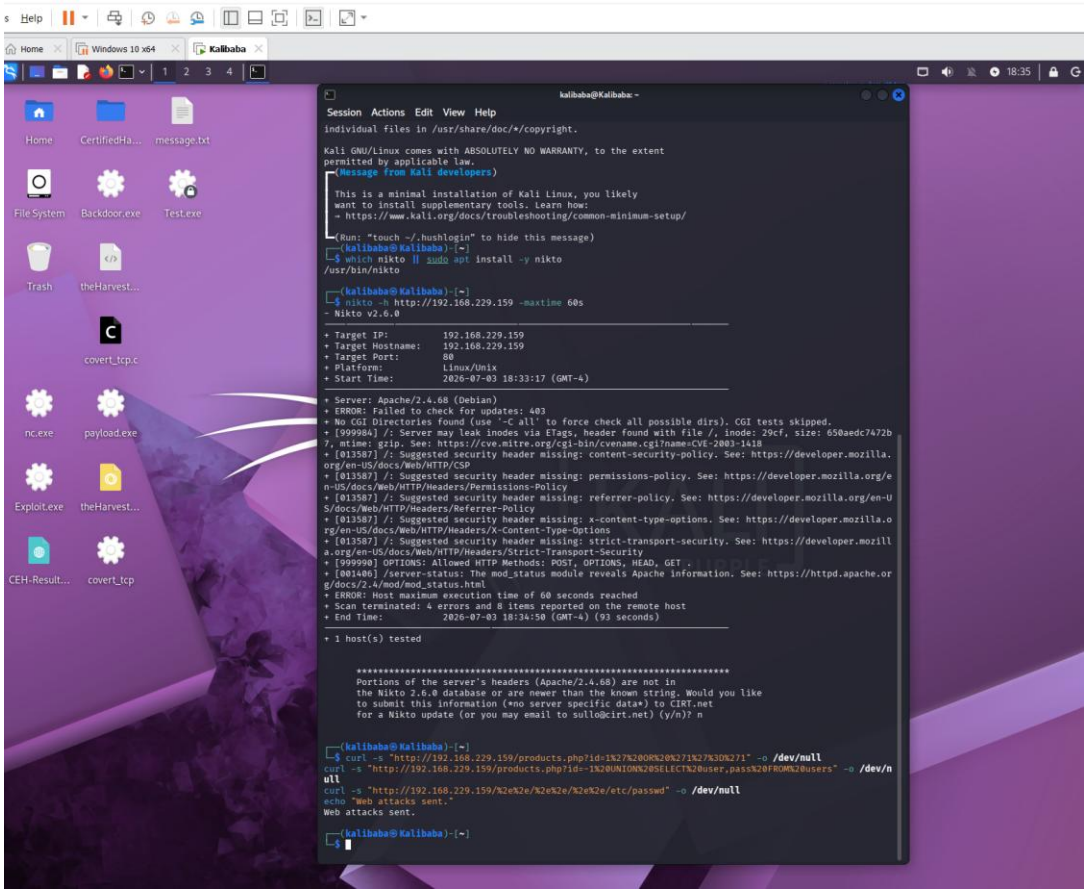


**Figure 5: SQL injection and path-traversal payloads delivered with curl**

## 7.4 Attack C — Port Scan from Windows (T1046)

Because nmap was not installed on the Windows 10 host, a native PowerShell TCP-connect sweep was used instead — a "living off the land" technique that requires no attacker tooling. The script was executed directly in the local PowerShell console on the Windows 10 VM (no remoting is involved); it simply opens outbound TCP connections to the Kali host. Ports 22 and 80 returned open, while the remaining ports were dropped and logged by UFW as blocks originating from 192.168.229.164.

```
$t="192.168.229.159"; $ports=22,80,20,21,23,25,53,110,135,139,143,
  443,445,993,1433,3306,3389,5900,8080,8443;
foreach($p in $ports){ $c=New-Object System.Net.Sockets.TcpClient;
  try{ $r=$c.BeginConnect($t,$p,$null,$null);
    if($r.AsyncWaitHandle.WaitOne(400) -and $c.Connected){
      Write-Host "Port $p : OPEN" } else {
      Write-Host "Port $p : blocked/closed" } }
  catch{ Write-Host "Port $p : blocked/closed" }
  finally{ $c.Close() } }
```

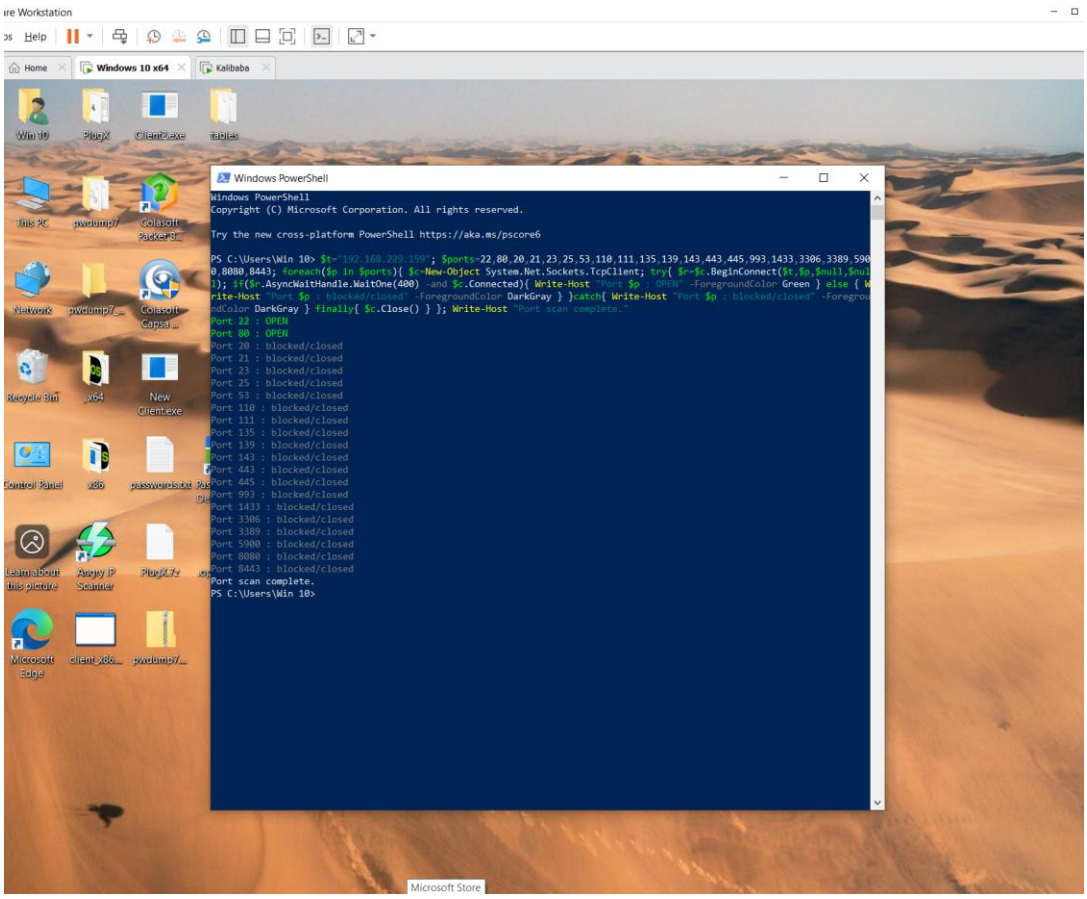


**Figure 6: Native PowerShell TCP port sweep from the Windows 10 host (192.168.229.164)**

# 8. Challenges Encountered and Solutions

Two substantive problems arose during testing. Both are common in real SIEM deployments and their resolution significantly strengthened the tool.

## 8.1 Empty access.log — Virtual-Host Logging Mismatch

**PROBLEM** On the first full run the SIEM reported *"no readable log found"* for Apache, even though the web attacks had clearly been executed. Investigation showed that `/var/log/apache2/access.log` was 0 bytes.

**ROOT CAUSE** Debian and Kali's default Apache configuration routes virtual-host traffic to `other_vhosts_access.log` using the `vhost_combined` format, which prefixes each line with an extra `vhost:port` field. The main access.log therefore stayed empty.

**FIX** The Apache source was pointed at the vhost log, and the parser's regular expression was extended to optionally skip the leading vhost field:

```
"apache": ["/var/log/apache2/other_vhosts_access.log",
           "/var/log/apache2/access.log"],

# regex gains an optional leading vhost:port group
r'^(?:\S+:\d+\s+)?(\d+\.\d+\.\d+\.\d+)\s+\S+ ...'
```

After the fix, Apache parsing jumped from 0 to 7,210 events, and all web detections fired correctly.

## 8.2 Alert Flooding — Deduplication of Scanner Traffic

**PROBLEM** Once Apache logs were read, a single nikto scan produced roughly **189 near-identical CRITICAL alerts** — one per probe URL — burying the genuinely important findings.

**ROOT CAUSE** The web-attack rule raised one alert per matching request. A scanner sends thousands of probes, so the rule generated thousands of duplicate alerts — the classic SIEM problem of *alert fatigue*.

**FIX** The rule was rewritten to **aggregate by (source IP, attack type)**, emitting a single alert per attack category with a request count and a representative sample. Alert volume dropped from 189 to a clean, analyst-friendly 14 — for example a single *"Path Traversal from 192.168.229.159 (×151 requests)"* line instead of 151 separate rows.

# 9. Results and Findings

The final SIEM run ingested logs from all three sources and produced the results summarised in Table 6.

| Metric | Value | Notes |
|---|---|---|
| Total events processed | 7,537 | Across SSH, Apache and firewall |
| Alerts generated | 14 | After deduplication |
| Critical / High alerts | 13 | Highest-priority findings |
| Unique source IPs | 7 | Including live and historical activity |
| MITRE techniques observed | 4 | T1110, T1046, T1190, T1595 (5 rules; T1110 twice) |

**Table 5: Summary metrics from the final SIEM run**

## 9.1 Log Sources Collected

| Source | Log File | Events |
|---|---|---|
| SSH | /var/log/auth.log | 89 |
| Apache | /var/log/apache2/other_vhosts_access.log | 7,210 |
| Firewall | /var/log/ufw.log | 238 |

**Table 6: Log sources and event counts**

## 9.2 Key Alerts

The correlation engine identified the full self-generated attack chain, plus historical brute-force activity present in the host's real logs (a useful demonstration that the parser works on genuine data):

- **CRITICAL — T1110:** Successful SSH login from 192.168.229.159 after 8 failures — the simulated account compromise.
- **CRITICAL — T1190:** Path Traversal (×151), Cross-Site Scripting (×268), Command Injection (×8), plus the two hand-crafted SQL injection payloads.
- **HIGH — T1046:** 192.168.229.164 blocked probing 11 different ports — the Windows port scan.
- **MEDIUM — T1595:** 192.168.229.159 generated 6,252 “404 Not Found” responses — the nikto directory-scanning behaviour.

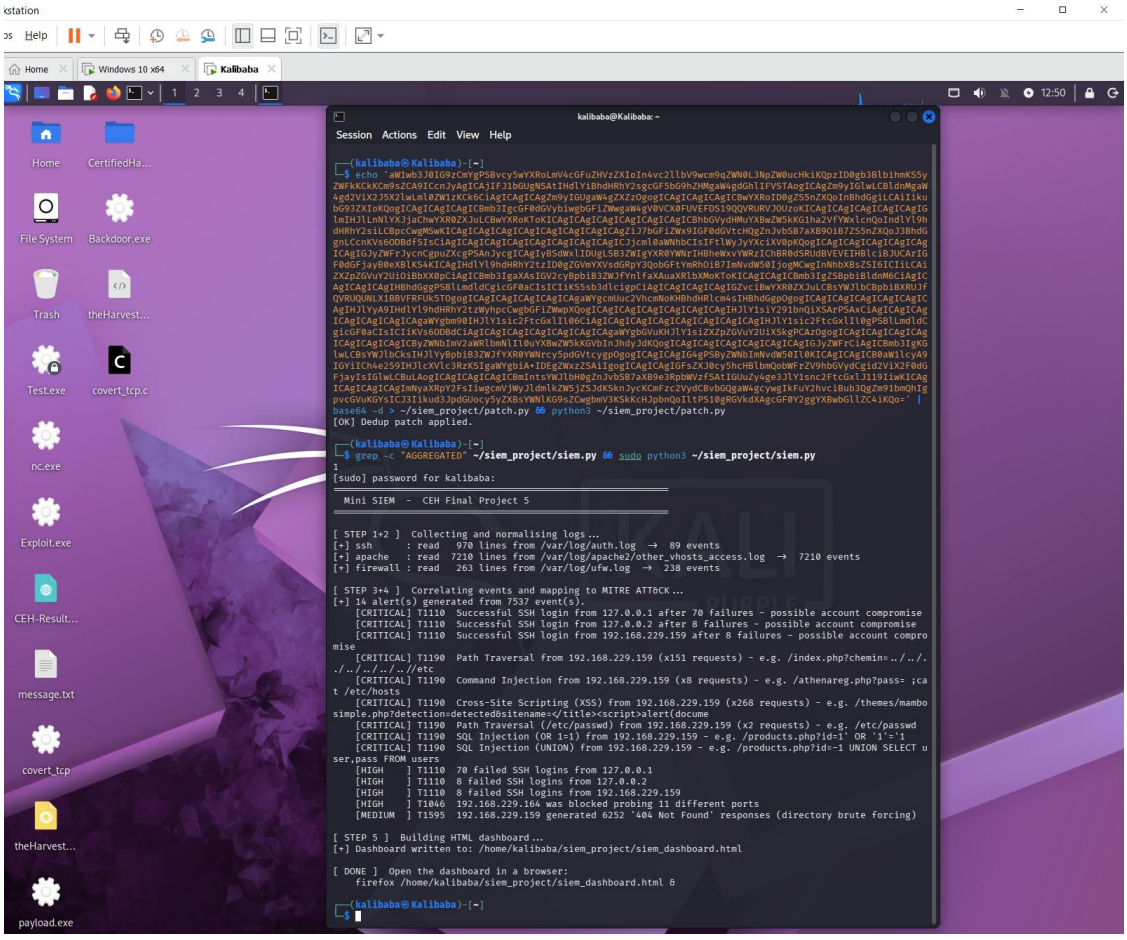

**Figure 7: SIEM execution — log collection, correlation and MITRE ATT&CK mapping output**

## 9.3 Dashboard

The HTML dashboard presents the results in a format suitable for a security operations context: summary metric cards, a colour-coded alert table with inline MITRE mapping and evidence, and supporting tables for log sources and top source IPs.

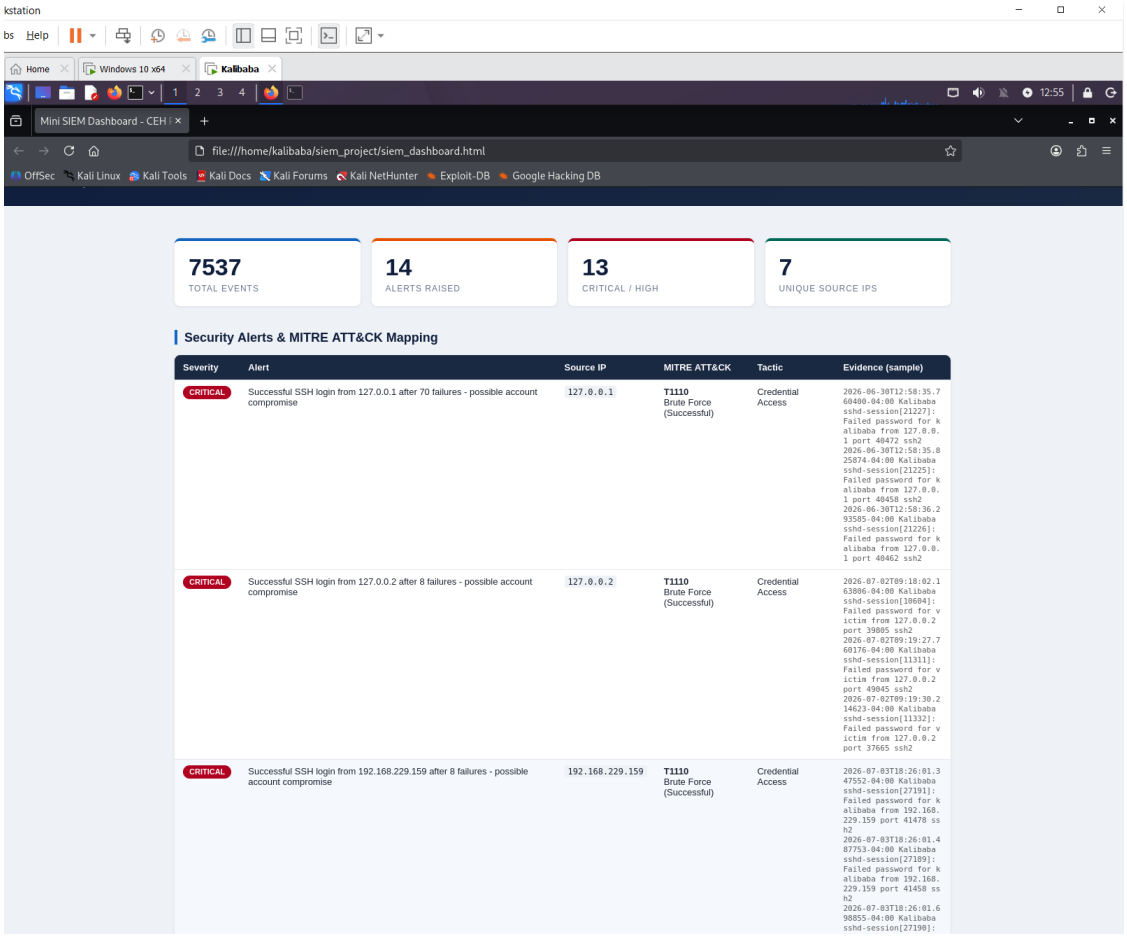


**Figure 8: Generated dashboard — summary metric cards and the MITRE-mapped security alert table**

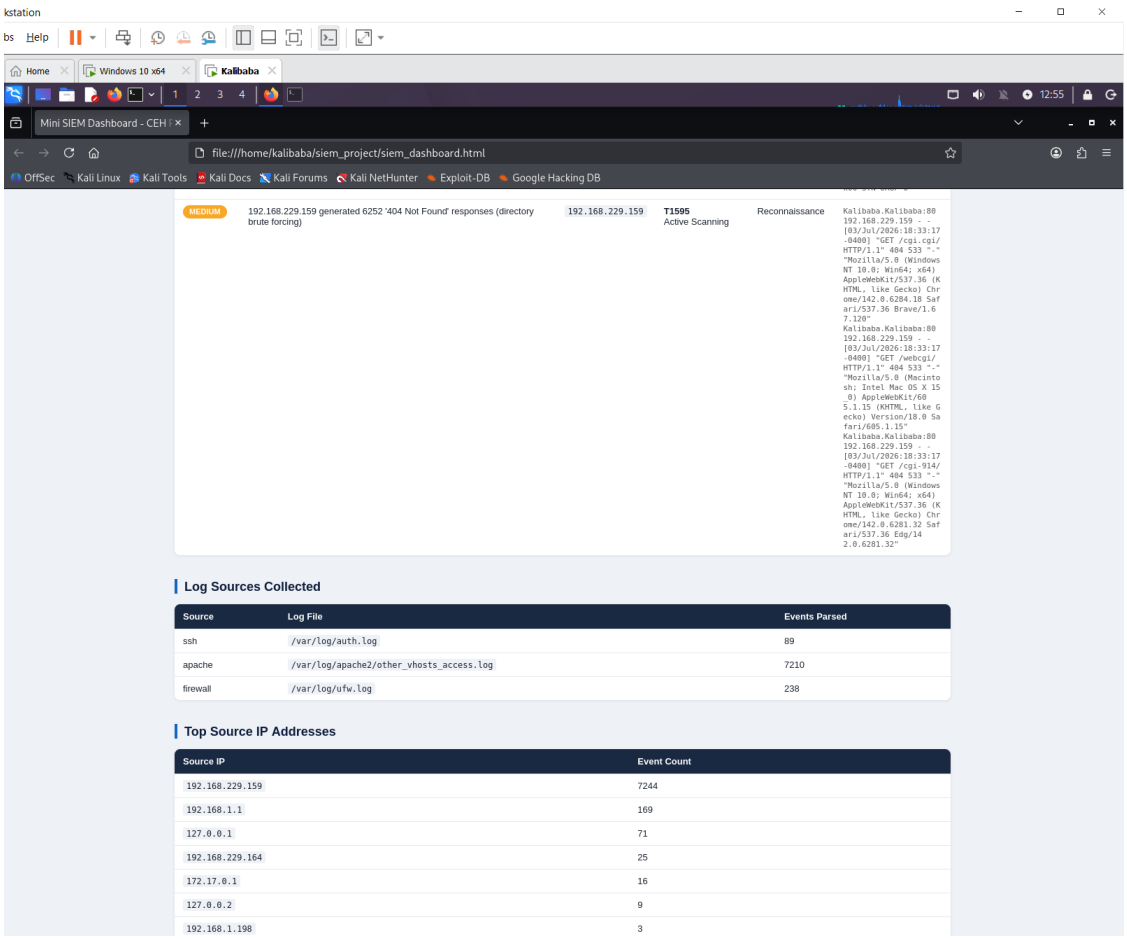


**Figure 9: Generated dashboard — log sources collected and top source IP addresses**

### 9.4 Limitations and Design Notes

Two design simplifications are worth stating explicitly, both of which are natural next steps toward a production system:

- **Threshold on cumulative counts, not time windows.** The correlation rules currently trigger on the total number of matching events from a source IP (for example "≥ 5 failed SSH logins"), not on a sliding time window. A rapid burst and a slow trickle are therefore treated the same. Adding a configurable time window (for example "≥ 5 failures within 60 seconds") is the primary planned enhancement and would reduce false positives from long-lived, low-rate activity.
- **On-demand analysis of static logs.** The tool reads the log files when run, rather than tailing them continuously. This is ideal for a lab investigation and for reproducible reporting, but a production deployment would stream events in real time and alert continuously.

Neither limitation affects the validity of the results in this project — every alert shown corresponds to genuine activity in the collected logs — but documenting them demonstrates an understanding of where this prototype sits relative to a full SIEM.

## 10. Defensive Recommendations

Based on the activity the SIEM detected, the following controls would reduce risk on the monitored host:

- Deploy fail2ban or equivalent to automatically block IPs after repeated SSH failures, complementing the SSH daemon's own rate-limiting.
- Enforce key-based SSH authentication and disable password logins to neutralise brute-force attacks entirely.
- Place a web application firewall (e.g. ModSecurity) in front of Apache to filter injection and traversal payloads before they reach the application.
- Forward all logs to a central collector so detection survives tampering or loss on the host itself.
- Alert on the “success-after-failures” pattern in real time, as it is the strongest single indicator of a completed credential attack.

## 11. Conclusion

This project produced a functional, dependency-free SIEM that collects, normalises, correlates and reports on real security events, and maps every finding to MITRE ATT&CK. Validated against a live three-stage attack chain, it processed 7,537 events and distilled them into 14 actionable, prioritised alerts. Just as importantly, the two problems encountered during development — the virtual-host logging mismatch and scanner-driven alert flooding — mirror genuine challenges faced in production security monitoring, and resolving them demonstrates practical understanding of how a SIEM behaves against real data rather than idealised samples.